



Practica 9

Practica9_2026.pdf

Ejercicio 1: ¿Explique claramente a qué se denomina excepción?

-Condicion inesperada o inusual que curre durante la ejecucion del programa y no puede ser manejada en el contexto local. Denota un comportamiento anomalo e indeseable que supuestamente ocurre raramente pero es neceasrio controlarlo, aunque ocurre con frecuencia

Ejercicio 2: ¿Qué debería proveer un lenguaje para el manejo de las excepciones? ¿Todos los lenguajes lo proveen?

-Un lenguaje que trate excepciones debe proveer:

- Un modo de definirlas
- Una forma de reconocerlas
- Una forma de lanzarlas y capturarlas
- Una forma de manejarlas especificando el codigo y respuestas
- Un criterio de continuacion

-No todos los lnguajes proveen un mecanismo completo para el manejo de excepciones. Los lenguajes mas antiguo pueden tener un enfoque mas limitado o directamente no proporcionarlo.Los lenguajes mas modernos si brindan este soporte

Ejercicio 3: ¿Qué ocurre cuando un lenguaje no provee manejo de excepciones? ¿Se podría simular? Explique cómo lo haría

-Cuando no provee un manejo de excepciones puede ser mas dificil de manejar errores de manera adecuada, los errores pueden propagarse y afectar al flujo del programa o provocar fallas.

-Se puede simular un mecanismo de manejo de excepciones utilizando tecnicas alternativas, como por ejemplo, estructuras de control condicionales con if-else y usar valores de retorno especiales para indicar errores. Por ejemplo:

- Se identifica la situacion excepcional
- Devolver valores de error especificos
- Validar valores de retorno
- Propagar manualmente el error
- Usar bloques try-catch simulados con if-else

Ejercicio 4: Cuando se termina de manejar la excepción, la acción que se toma luego es importante. Indique

1. ¿Qué modelos diferentes existen en este aspecto?

-Reasuncion → Se refiere a la posibilidad de retomar la ejecucion del programa despues de manejar una excepcion. El controlador de excepciones realiza las acciones necesarias para manejar la excepcion y luego el programa continua su ejecucion a partir del punto donde se produjo la excepcion

-Terminacion → El controlador de excepciones realiza las acciones necesarias para manejar la excepcion pero no se retorna al punto donde se produjo la excepcion. Continua su ejecucion a partir de la finalizacion del manejador

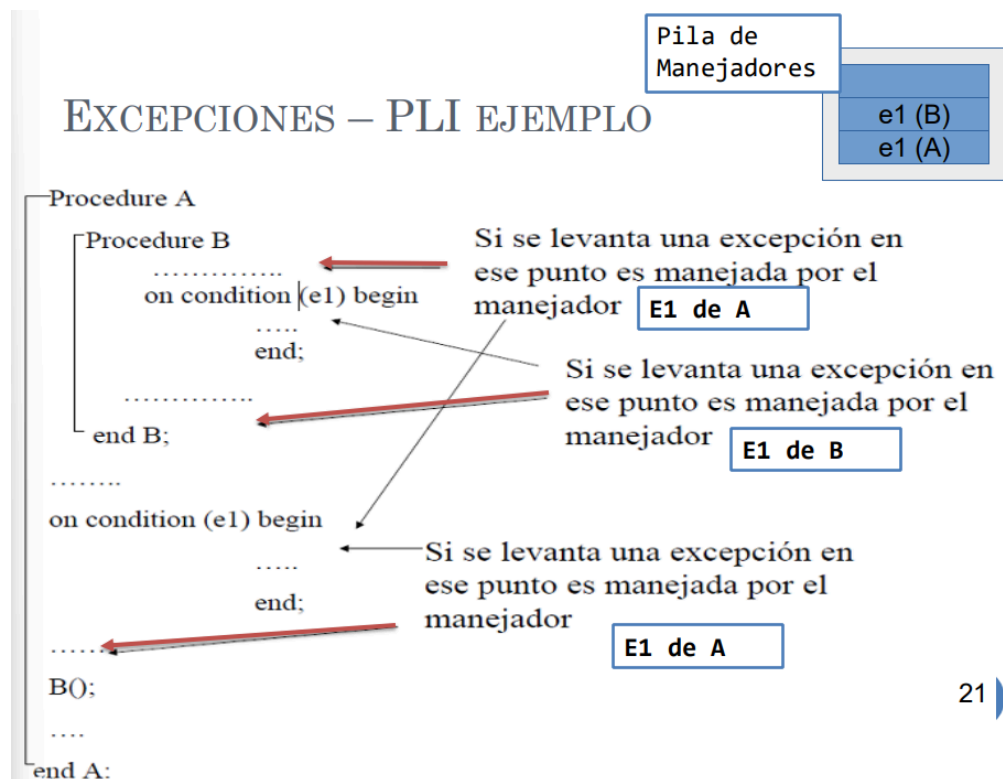
2. Dé ejemplos de lenguajes que utilizan cada uno de los modelos presentados anteriormente. Por cada uno responda respecto de la forma en que trabaja las excepciones.

- ¿Cómo se define?**
- ¿Cómo se lanza?**
- ¿Cómo se maneja?**
- ¿Cuál es su criterio de continuación?**
- ¿Cuál de esos modelos es más inseguro y por qué?**

-Reasuncion:

▼ PL/I

- Primer lenguaje que incorporo el manejo de excepciones y utiliza el criterio de reasuncion
- Las excepciones se llaman CONDITIONS y los manejadores se declaran con la sentencia ON
 - ON CONDITION (Nombre-Excepcion) Manejador
- El manejador puede ser una instruccion o un bloque (entre begin y end)
- Las excepciones se lanzan explicitamente con la palabra clave SIGNAL
 - SIGNAL CONDITION (Nombre-Excepcion)
- Tiene una serie de excepciones predefinidas con su manejador asociado (EJ zerodivide, que se le vanta cuando hay division por 0)
- Los manejadores se ligan dinamicamente con las excepciones. Una excepcion siempre esta ligada con el ultimo manejador definido
- El alcance de un manejador termina cuando finaliza la ejecucion de la unidad donde fue declarado



-Terminacion:

▼ ADA

-Utiliza criterio de Terminacion

-Las excepciones se definen en la zona de definicion de variables y tienen el mismo alcance en la unidad → MiExcepcion: exception;

-La lista de controladores de excepciones lleva el prefijo de la palabra clave exception.Cada controlador lleva el prefijo de la palabra clave when seguido de las acciones

-Se puede utilizar when others para capturar cualquier excepcion no especificad. (Se debe colocar al final del bloque de manejo de excepciones; Posee efectos colaterales)

-Las excepciones se levantan explicitamente con la palabra clave raise

-Los manejadores pueden agregarse y encontrarse al final de diferentes unidades de programa

▼ Tiene excepciones predefinidas:

-Constraint_Error → se intenta violar una restriccion impuesta en la declaracion (acceder mas alla de los limites de un array)

-Program_Error → se intenta violar la estructura de control o regla de lenguaje (funcion sin devolver un valor)

-Storage_Error → se hace una violacion de memoria (se requiere mas memoria disponible)

-Tasking_Error → error de comunicacion y manejo de tareas (concurrency y programacion de tareas)

-Name_Error → error de nombre (al intentar abrir un fichero que no existe)

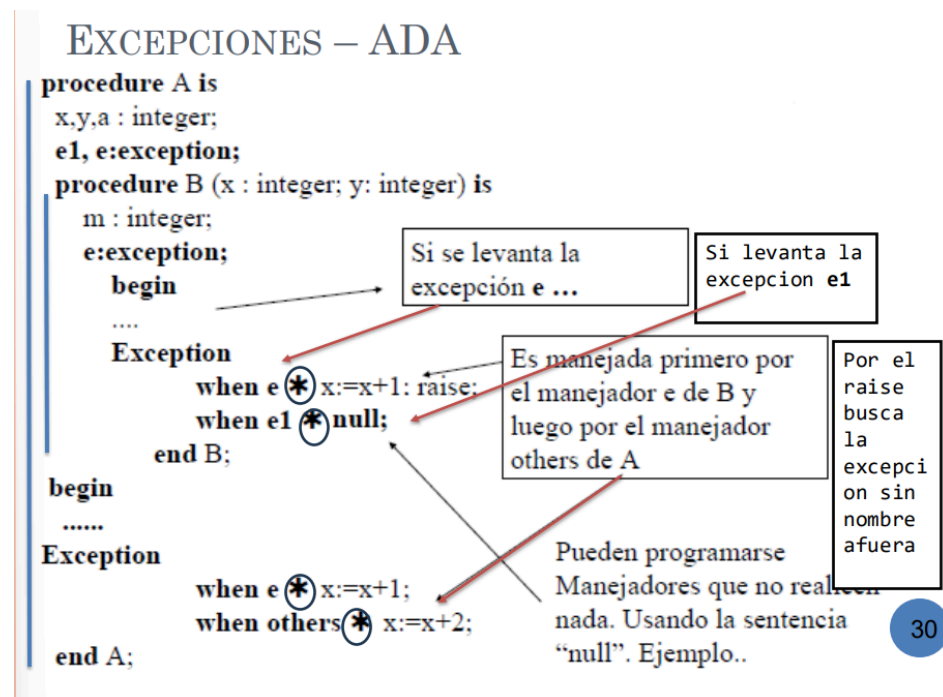
-Propagacion:

-Si la unidad que genera la excepcion proporciona un manejador para la misma el control se transfiere inmediatamente a ese manejador:

-Se omiten acciones que siguen al punto en el que se genero la excepcion

- Se ejecuta el manejador
- El programa continua su ejecucion normalmente, desde la instruccion que sigue al manejador
- Si la unidad no proporciona un manejador se busca por propagacion, dinamicamente se busca un manejador:
 - Se termina la unidad donde se produce la excepcion
 - Si el manejador no encuentra en ese lugar, se propaga dinamicamente y busca en quien lo llamo
 - Siempre tener en cuenta el alcance, puede convertirse en anonima. Al propagarse a otras unidades la variable excepcion declarada ya no esta en el alcance y quedara sin nombre, entrara por when others
- Raise lanza excepciones que pueden ser definidas por el programador. Se puede hasta levantar nuevamente colocando solo la palabra raise y levanta la misma excepcion que se esta manejando

▼ EJEMPLOS



EXCEPCIONES – ADA – EJEMPLO

```

Prog MAIN
e: exception
...
Begin
...
.. if (condError) then
    raise e
  endif
....
Exception
when e : begin ...
    ... (raise)
End
When others Begin
    ... ..
End
End //(de main)
  
```

*****DECLARACION**

***** INVOCACION**
 ** si fuera sólo (raise) levanta una excepción anónima afuera

*****MANEJADORES**
 *** este raise levanta la misma excepción que está manejando, pero como es por terminación Saldría por propagación y terminaría dando error

Tener en cuenta :

- La asociación de la excepción con el manejador es **determinístico** (*variable ↔ manejador de esa variable*). Asigno mismo nombre a ambos
- Si se deseara **continuar ejecutando las instrucciones de un bloque** donde se lanza una excepción, es preciso “**crear un bloque más interno**”. Se usa **Declare** para **amar una unidad**, como se muestra en el ejemplo y luego **agregar instrucciones restantes abajo**. (simular reasunción)

Procedure Bloque () is

```

....
begin
...
  Declare
  .....
  .....
  begin
    exception
    Manejadores
    .....
  end; -- del bloque interno
....
  
```

instrucción declare se utiliza para declarar variables locales, instrucciones, excepciones dentro de un bloque, y armar una unidad interna

bloque interno para manejar las instrucciones que pueden fallar

En esta sección se deben colocar Instrucciones que es preciso ejecutar, aunque se haya levantado la excepción en el bloque interno

32

end;

▼ C++

- Utiliza criterio de Terminacion. Tiene excepciones estandar predefinidas
- Try → indica los bloques donde pueden llegar a levantarse excepciones
- Catch → es utilizada para especificar los manejadores, Catch(NombreExcepcion)

- Las clausulas catch deben estar despues del bloque try y antes de cualquier codigo que este fuera del bloque try. Los manejadores van asociados a bloques { }
- Throw → Se utiliza para lanzar explicitamente una excepcion
- Funcionamiento:
 - El bloque try contiene el codigo que puede lanzar una excepcion
 - Si se lanza una excepcion el control se transfiere inmediatamente a la clausula catch
 - Si la excepcion coincide con el tipo especificado en la clausula catch, se ejecuta el bloque de codigo de esa clausula catch.
 - Si la excepcion se maneja exitosamente, la ejecucion continua despues del bloque try-catch
 - Si no se encuentra un catch correspondiente o no se maneja la excepcion, la excepcion puede propagarse hacia try-catch externos. Sino puede resultar en una finalizacion abrupta del programa
- Las rutinas en su interface pueden listar las excepciones que ellas pueden alcanzar
 - void rutina () throw (Ayuda, Zerodivide);
 - Que pasa si la rutina?:
 - Alcanza otra excepcion que no esta contemplada en el listado de la Interface:
 - No se propaga la excepcion y uan funcion especial se ejecuta automaticamente: unexpeted() que generalmente causa abort(), provocando el final del programa. unexpected puede ser redefinida por el programador
 - Coloca en su interface el listado de posibles excepciones a alcanzar:
 - Se propaga la exepcion. Si una excepcion es repetidamente propagada y no matchea con ningun

manejador, entonces una funcion terminate() se ejecuta automaticamente

-Coloca en su interface una lista vacia (throw())?:

-Significa que ninguna excepcion sera propagada

▼ CLU

-Utiliza el criterio por Terminacion

-Solamente pueden ser alcanzadas por procedimientos. Si una instruccion genera una excepcion, el procedimiento que contiene la instruccion retorna anormalmente al generar la excepcion. Un procedimiento no puede manejar una excepcion generada por su ejecucion, quien llama al procedimiento debe encargarse de manejarla

-Las excepciones que un procedimiento puede lanzar se declaran en su encabezado. Se alcanzan explicitamente con la palabra clave signal

-Los manejadores se colocan al lado de una sentencia simple o compleja y llevan la palabra clave when

-Posee excepciones predefinidas por el manejador y se pueden pasar parametros a los manejadores

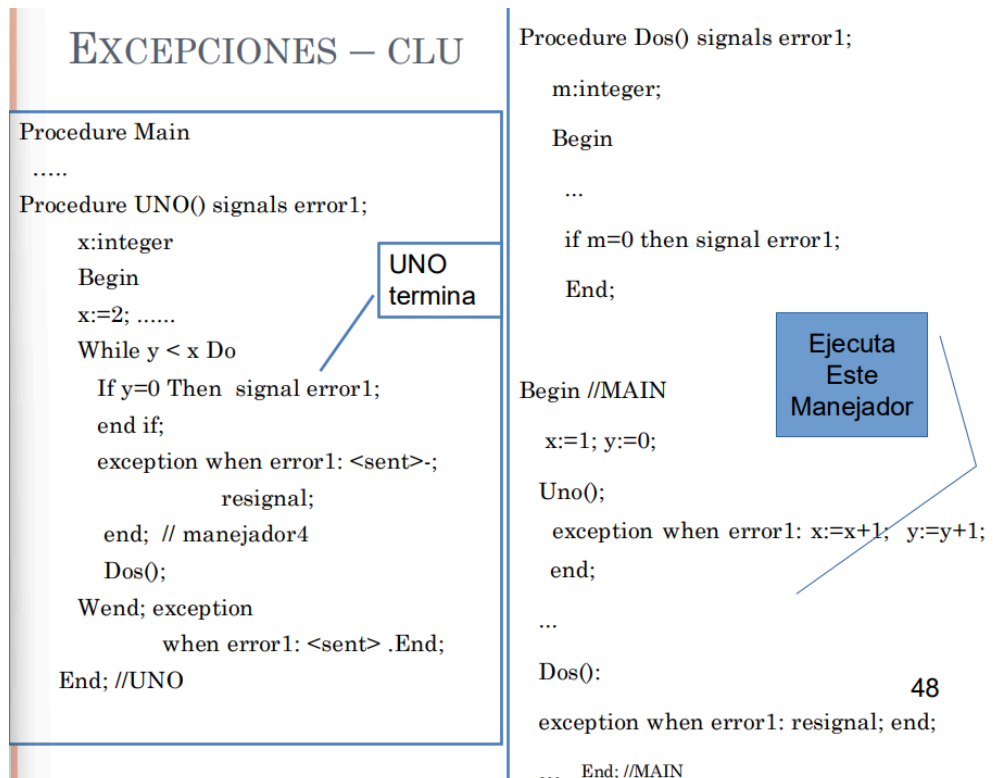
-Se pueden volver a levantar una sola vez utilizando la palabra clave resignal en cualquier parte del codigo

```
Procedure ejemplo () signals e1, e2
Begin
.....
if ( .. ) then A(); except
    when e1: Manejador1;
    when e2 : Manejador2;
    when others : Manejador3;
end;

....
End;
```

-Propagacion:

- Se termina el procedimiento donde se levanto la excepcion y devuelve el control al llamante inmediato donde se debe encontrar el manejador (UNA VEZ)
- Si el manejador se encuentra en ese ambito, se ejecuta y luego se pasa el control a la sentencia siguiente a la que esta ligado dicho manejador
- Si el manejador no se encuentra en ese lugar, la excepcion se propaga estaticamente en las sentencias asociadas. Se repite el proceso para las sentencias incluidas estaticamente
- En caso de no encontrar ningun manejador en el procedimiento que hizo la llamada se levanta una excepcion failure y devuelve el control, terminando el programa



▼ Java

- Las excepciones son objetos que pueden ser alcanzados y manejados por bloques de excepciones
- Cada excepcion esta representada por una instancia de la clase Throwable o una de sus subclases (Error y Excepcion)
- La gestion de excepciones se lleva a cabo mediante cinco palabras clave: try, catch, throw, throws y finally

- Se debe especificar mediante la clausula throws cualquier excepcion que se envia desde un metodo
- Se debe poner cualquier codigo que el programador desee que se ejecute siempre en el metodo finally

FASES DEL TRATAMIENTO DE EXCEPCIONES

- **Detectar e informar del error:**
 - Lanzamiento de Excepciones → throw
 - Un método detecta una condición anormal que le impide continuar con su ejecución y finaliza “lanzando” un objeto Excepción.
- **Recoger el error y tratarlo:**
 - Captura de Excepciones → bloque try-catch
 - Un método recibe un objeto Excepción que le indica que otro método no ha terminado correctamente su ejecución y decide actuar en función del tipo de error.

EXCEPCIONES - JAVA

```
try{
    ...
}
Catch (ArithmeticException e)
    { trataArithmeticException(e);}
catch (Exception e2)
    { trataTodasLasExcepciones(e2); }
Finally
    { bloque final; }
```

Orden correcto

Puede estar o no
Si está, la ejecución de su código se realiza cuando se termina la ejecución del bloque Try, se haya o no levantado una excepción.

Propagación,
bloques Try
anidados

```
// demuestra cómo lanzar una excepción cuando ocurre una división entre
// cero
public int cociente( int numerador, int denominador )
    throws ArithmeticException
{
    return numerador / denominador;
}
```

52

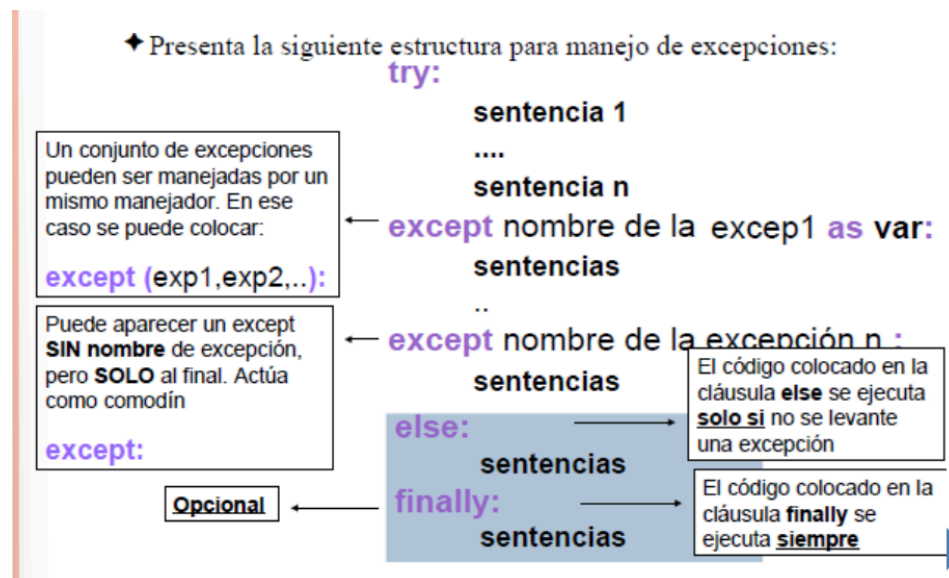
Veamos un ejemplo en [compilador online \(java\)](#)

▼ Python

- Se maneja a traves de bloques try-except

-La declaracion try funciona de la siguiente manera:

- Primero se ejecuta el bloque try
- Si no ocurre ninguna excepcion, el bloque except se saltea y termina la ejecucion de la declaracion try
- Si ocurre una excepcion durante la ejecucion del bloque try, el resto del bloque se saltea. Luego si su tipo coincide con la excepcion nombrada luego de la palabra reervada except, se ejecuta el bloque except y la ejecucion continua luego de la declaracion try
- Si ocurre una excepcion que no coincide con la excepcion nombrada en el except, esta se pasa a declaraciones try de mas afuera, si no se encuentra nada que la maneje, entonces es una excepcion no manejada y la ejecucion frena con un mensaje de error
- En el caso que no se haya levantado ninguna exepcion existe el bloque else (opcional) y se ejecutan las instrucciones de este bloque



EXCEPCIONES - PYTHON

¿Qué sucede cuando una excepción no encuentra un manejador en su bloque “try except”?

- **Busca estáticamente**

Analiza si ese try está contenido dentro de otro y si ese otro tiene un manejador para esa excepción.
Sino...

- **Busca dinámicamente**

Analiza quién lo llamó y busca allí

- **Si no se encuentra un manejador, se corta el proceso y larga el mensaje standard de error**

- **Levanta excepciones explícitamente con “raise”**

55

▼ PHP

-Usa modelo de terminacion

-Una excepcion puede ser lanzada (thrown) y atrapada (catched).
El codigo esta dentro de un bloque try

-Cada bloque try debe tener al menos un bloque catch correspondiente

-Las excepciones pueden ser lanzadas o relanzadas dentro de un bloque catch. Se puede usar el bloque finally luego de los bloques catch

-El objeto lanzado debe ser una instancia de la clase Exception o de una subclase de exception, en caso contrario lanzara Error Fatal de PHP

-Cuando una excepcion es lanzada, el codigo siguiente a la declaracion no sera ejecutado, PHP intentara encontrar el primer bloque catch coincidente. Si una excepcion no es capturada se emite Error Fatal

EXCEPCIONES EN PHP

```
<?php
function inverse($x) {
    if (!$x) {
        throw new Exception('División por cero.');
```

El resultado del ejemplo sería:

```
0.2
Primer finally.
Excepción capturada: División por cero.
Segundo finally.
Hola Mundo
```

59

Ejercicio 5: La propagación de los errores, cuando no se encuentra ningún manejador asociado, no se implementa igual en todos los lenguajes. Realice la comparación entre el modelo de Java, Python y PL/I, respecto a este tema. Defina la forma en que se implementa en un lenguaje conocido por Ud.

Lenguaje	Tipo de Propagación	Comportamiento si no se maneja	Características Específicas
Java	Dinámica (pila de llamadas)	Terminación con traza de pila	Excepciones comprobadas (<code>throws</code>), propagación hasta <code>main</code> , mensaje de error estándar.
Python	Híbrida (estática en bloques <code>try</code> anidados, luego dinámica)	Terminación con traza de pila	Búsqueda estática y dinámica, sin declaración de excepciones, mensaje de error estándar.
PL/I	Limitada (reasunción)	Comportamiento indefinido o terminación (depende del entorno)	Enfocado en manejo local, manejadores dinámicos, comportamiento depende del compilador.
JavaScript	Dinámica (pila de llamadas)	Error en consola (navegadores) o	Propagación simple, sin excepciones comprobadas,

Lenguaje	Tipo de Propagación	Comportamiento si no se maneja	Características Específicas
		terminación (Node.js)	comportamiento varía según el entorno.

Ejercicio 6: Sea el siguiente programa escrito en Pascal

```

...
Procedure Manejador;
  Begin ... end;
Procedure P(X:Proc);
  begin
    ....
    if Error then X;
    ....
  end;
Procedure A;
  begin
    ....
    P(Manejador);
    ....
  end;
....

```

¿Qué modelo de manejo de excepciones está simulando? ¿Qué necesitaría el programa para que encuadre con los lenguajes que no utilizan este modelo? Justifique la respuesta.

-El modelo de manejo de excepciones que está simulando sería el de reanudación, ya que cuando se detecta un error se invoca al manejador X y luego la ejecución continúa al punto donde ocurrió el error, básicamente, continuando la ejecución normal.

-Para que el programa encuadre con los lenguajes que no utilizan este modelo necesitaría:

-Interrupción/Propagación:

-En lugar de continuar la ejecución tras invocar al manejador, el programa debería interrumpir el flujo y propagar el error a un nivel

superior. Para eso te necesitaría una forma de poder salir del procedimiento actual y transferir el control al llamador

-Es decir, se necesita un exit o break o mecanismo de salto para indicar que el error hay que manejarlo afuera del procedimiento actual

-Definición de un bloque de manejo externo:

-El manejo de excepciones ocurre en un bloque separado, por ejemplo, try catch o exception en otros lenguajes. El lenguaje debería reestructurarse para que el manejador se ejecute como parte de un bloque de manejo en lugar de ser llamado directamente dentro del procedimiento

-Simular Excepciones:

-Pascal no tiene excepciones nativas por lo que se simulan con la condición del IF. Para emular el modelo de terminación se podría usar una variable global o un parámetro de retorno para indicar que ocurrió un error, y el llamador decidirá cómo manejarlo

Ejercicio 7: Sea el siguiente programa escrito en Pascal:

<pre>Program Principal; var x:int; b1,b2:boolean; Procedure P (b1:boolean); var x:int; Procedure Manejador1 begin x:=x + 1; end; begin x:=1; if b1=true then Manejador1; x:=x+4; end; Procedure Manejador2; begin x:=x * 100; end;</pre>	<pre>Begin x:=4; b2:=true; b1:=false; if b1=false then Manejador2; P(b); write (x); End.</pre>
---	--

1. Implemente este ejercicio en PL/1 utilizando manejo de excepciones

```
Prog Principal
DCL x DEC FIXED (3,2) INIT(0);
var x: int; b1,b2: boolean
```

```

Proc P
  ON CONDITION (Manejador1) begin
    x = x + 1;
  end;
  x = 1;
  if (b1 = true) then SIGNAL CONDITION Manejador1
  x = x + 4;
  ON CONDITION (Manejador2) begin
    x = x * 100;
  end;
end;

Begin
  x = 4;
  b2 = true;
  b1 = false;
  ON CONDITION Manejador2 begin
    if b1 = false then SIGNAL CONDITION Manejador2;
    P
    PUT SKIP LIST(x)
  end;
end;

```

2. **¿Podría implementarlo en JAVA utilizando manejo de excepciones? En caso afirmativo, realícelo.**

-Java utiliza Terminacion, por lo que implementar reasuncion no seria posible

Ejercicio 8: Sean los siguientes, procedimientos de un programa escrito en JAVA:


```

public static void main (String[] args){
    Double array_doubles[] = new double[500];
    for (int i=0; i<500; i++){
        array_doubles[i]=7*i;
    }
    for (int i=0 ; i<600 ; i=i+25){
        try{
            system.out.println("El elemento en " + i + " es " + acceso_por_indice (array_doubles,i));
        }
        catch (ArrayIndexOutOfBoundsException e){
            system.out.println(e.toString());
        }
        catch (Exception a){
            system.out.println(a.toString());
        }
        finally{
            system.out.println("sentencia finally");
        }
    }
}

Public static double acceso_por_indice (double [] v, int indice) throws Exception; ArrayIndexOutOfBoundsException{
    if ((indice>=0) && (indice<v.length)){
        Return v[indice];
    }
    else{
        if (indice<0){
            // caso excepcional
            Throw new ArrayIndexOutOfBoundsException(" el índice" + indice + " es un número negativo");
        }
    }
}

```

Hoja 2 de 7

Conceptos y Paradigmas de lenguajes de Programación 2026

```

    }
    else{
        // caso excepcional
        Throw new Exception(" el índice" + indice + " no es una posición válida");
    }
}

```

1. Analizar el ejemplo y decir qué manejadores ejecuta y en qué valores quedan las variables. JUSTIFIQUE LA RESPUESTA.

-La variable *i* toma valores, 0,25,50,75 hasta el 575. El arreglo "array_doubles" tiene un tamaño de 500 elementos. Cuando *i* > 500 el índice está fuera del rango del array y el método "acceso_por_indice" lanza una Excepción genérica "el índice no es una posición válida" en lugar de ArrayIndexOutOfBoundsException

-Cuando *i* < 500 → No se lanza ninguna excepción y se accede correctamente a las posiciones del vector

-Se imprime:

-El elemento en *i* (0), es (0.0), y "sentencia finally" del bloque finally

-...

-El elemento en i (475), es (3325.0), y "sentencia finally" del bloque finally

-Cuando i > 500 (500, 525, 550, 575) → Indice fuera de rango, se lanza Excepcion generica desde acceso_por_indice

-Se imprime:

-java.lang.Exception: El indice 500 no es una posicion valida y luego la sentencia finally del bloque main

-java.lang.Exception: El indice 525 no es una posicion valida y luego la sentencia finally del bloque main

-java.lang.Exception: El indice 550 no es una posicion valida y luego la sentencia finally del bloque main

-java.lang.Exception: El indice 575 no es una posicion valida y luego la sentencia finally del bloque main

2. La excepción se propaga o se maneja en el mismo método? ¿Qué instrucción se agrega para poder propagarla y que lleve información?.

-La excepcion se propaga al metodo acceso_por_indice. Donde en este metodo se lanza explicitamente con la sentencia Throw new Excepcion y el metodo main la captura

3. como modificaría el método "acceso_por_indice" para que maneje él mismo la excepción.

```
public static void main (String[] argos){
    Double array_doubles[] = new double[500];
    for (int i=0; i<500; i++){
        array_doubles[i]=7*i;
    }
    for (int i=0; i < 600 ; i++){
        System.out.println("El elemento en " + i + " es"
            + acceso_por_indice(array_doubles, i ));
    }
}
```

```

public static double acceso_por_indice(double [] v, i
nt indice) {
    try {
        if (indice >= 0 && indice < v.length) {
            return v[indice];
        } else if ( indice < 0 ) {
            throw new ArrayIndexOutOfBoundsException
("El indice" + indice
            + "es negativo"_
        } else {
            throw new Exception("El indice" + indice
+ "no es una pos valida")
        }
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Excepcion atrapada en
acceso_por_indice:" +
            e.getMessage());
        return -1;
    } catch (Exception e){
        System.out.println("Exepcion general en a
cceso_por_indice:" +
            e.getMessage());
        return -1;
    }
}
}

```

Ejercicio 9: Indique diferencias y similitudes entre Phyton y Java con respecto al manejo de excepciones.

-Similitudes:

- Try-catch para envolver el codigo que maneja excepciones y bloques (catch en Java, except en Python)
- Las excepciones se organizan en una jerarquia con clases (Throwable en Java, Exception en Python) de la cual derivan toras excepciones especificas y subclasses

- Permiten capturar multiples tipo de excepciones en un solo bloque o en bloques separados
- Bloque finally, siempre se ejecuta independientemente de si se lanza una excepcion o no
- Si no se captura una excepcion, se propaga hacia arriba en la pila de llamadas

-Diferencias:

- Los bloques envolventes en Java se escriben como try-catch-finally y en Python como try-except-else-finally
- Java distingue entre excepciones verificadas (checked, debe declararse con throws) y no verificadas (unchecked, derivadas de RuntimeException) ; Python no tiene excepciones verificadas, todas son no verificadas
- En Java las excepciones verificadas se declaran en la firma del metodo con la palabra clave throws; Python no requiere declarar excepciones en la firma de una funcion
- Python tiene un bloque else que se ejecuta si no atrapas ninguna excepcion en los try, Java no tiene esto y se coloca en un codigo despues del bloque catch pero no es tan explicito
- Java requiere especificar el tipo de excepcion; Python puede omitir esto aunque no es una buena practica
- Java tiene excepciones personalias extendiendo Exception o RuntimeException; Python las tienen extendiendo de Exception

Ejercicio 10: ¿Qué modelo de excepciones implementa Ruby?. ¿Qué instrucciones específicas provee el lenguaje para manejo de excepciones y cómo se comportan cada una de ellas?

- Ruby usa manejo de excepciones basado en objetos con criterio tipo terminal. Este modelo permite interrumpir el flujo normal de ejecucion del programa cuando ocurre un error o una condicion excepcional, transfiriendo el control a un bloque de codigo disenado para manejar la situacion.
- Cuando se lanza una excepcion con raise y no se captura en un bloque rescue, esta se propaga hacia el metodo o bloque llamador donde ocurrio el

error. La búsqueda comienza en el bloque `begin` más cercano y continúa hacia arriba en la pila de llamadas hasta encontrar un `rescue` que coincida con el tipo de excepción o hasta llegar al nivel superior del programa

-Si no se encuentra un manejador, el programa termina con un mensaje de error. Ruby no distingue entre excepciones verificadas y no verificadas ya que se propagan todas de la misma forma

-Manejo de Excepciones en Ruby (Instrucciones):

1. `begin`

- Inicia un bloque donde se espera una excepción; el código se ejecuta hasta que ocurre un error, luego Ruby busca un `rescue` coincidente.
- Implícito en métodos si no se usa explícitamente.

```
begin
  result = 10 / 0
rescue ZeroDivisionError
  puts "No se puede dividir por cero"
end
```

2. `rescue`

- Captura excepciones específicas o genéricas (`StandardError` por defecto), asignándolas opcionalmente a una variable.
- Compara la excepción con las clases especificadas y ejecuta el bloque correspondiente; si no hay coincidencia, la excepción se propaga.

```
begin
  raise ArgumentError, "Argumento inválido"
rescue ArgumentError => e
  puts "Error capturado: #{e.message}"
end
```

3. `else`

- Ejecuta código si no ocurre ninguna excepción en el bloque `begin`.

```
begin
  puts "Ejecutando código sin errores"
rescue StandardError => e
  puts "Error: #{e.message}"
else
  puts "No hubo errores"
end
```

4. `ensure`

- Ejecuta código siempre, con o sin excepción.
- Ideal para liberar recursos.
- Se ejecuta incluso con `return` o errores no manejados.

```
begin
  puts "Abriendo archivo"
  raise "Error inesperado"
rescue => e
  puts e.message
ensure
  puts "Cerrando archivo"
end
```

5. `raise`

- Lanza una excepción manualmente (`RuntimeError` por defecto o un tipo específico con mensaje).
- Puede relanzar la excepción actual si se usa sin argumentos.

```
def dividir(a, b)
  raise ArgumentError, "El divisor no puede ser cero"
if b == 0
  a / b
end

begin
  dividir(10, 0)
rescue ArgumentError => e
```

```
puts e.message
end
```

6. `retry`

- Reinicia el bloque `begin` desde el principio tras manejar una excepción en `rescue`.
- Útil para reintentos, pero debe usarse con cuidado para evitar bucles infinitos.

```
intentos = 0

begin
  intentos += 1
  raise "Error simulado" if intentos < 3
  puts "Éxito después de #{intentos} intentos"
rescue StandardError
  retry if intentos < 3
end
```

7. `throw` y `catch`

- `catch` define un bloque etiquetado.
- `throw` interrumpe ese bloque con un valor opcional.
- Usado para control de flujo ligero, no para manejo de errores.

```
resultado = catch(:done) do
  throw(:done, "Terminado") if true
  "No se ejecuta"
end

puts resultado
# => "Terminado"
```

Ejercicio 11: Indique el mecanismo de excepciones de javascript.

-Se lanza cualquier tipo de excepcion usando throw y se manejan usando declaraciones try-catch

-Tipo de excepciones:

-Casi cualquier objeto se puede lanzar en JS, sin embargo no todos los objetos se lanzan igual

-Excepciones ECMAScript

-Interfaz DOMException representa un evento anormal que ocurre como resultado de llamar a un método o acceder a una propiedad de una API web y la interfaz DOMError describe un objeto error que contiene nombre de error

-Expresión throw:

-Se utiliza para lanzar una excepción, especifica el valor que se lanzará
throw expression;

-Es posible lanzar cualquier expresión y especificar un objeto, no solo expresiones de un tipo específico

-Declaración try-catch:

-Consta de un bloque try, que tiene una o más declaraciones, y un bloque catch que contiene declaraciones que especifican qué hacer si se lanza una excepción en el bloque try

-Si alguna de las instrucciones del bloque try lanza una excepción, el control inmediatamente cambia al bloque catch. Si no se lanza ninguna excepción, se omite el bloque catch. El bloque finally se ejecuta siempre después de que se ejecutan los bloques try y catch pero antes de las declaraciones que siguen a la declaración try-catch

-Bloque catch:

-Se puede usar para manejar todas las excepciones que se pueden manejar en un bloque try

-El bloque catch especifica un ID que contiene el valor especificado por la expresión throw. Puede usar este ID para obtener información sobre la excepción lanzada

-Bloque finally:

-Se ejecuta después de los bloques try y catch

-Es importante notar que se ejecutara independientemente de que se produzca una excepcion, sin embargo, si se lanza una las declaraciones del bloque finally se ejecutan incluso si ningun bloque catch maneja la excepcion que se lanzo

-Anidamiento:

- Se pueden anidar uno o mas declaraciones try catch
- Si el bloque try interno no tiene un bloque catch correspondiente:
 - Se debe contener un bloque finally y
 - El bloque catch adjunto de la declaracion try-catch se comprueba para una coincidencia

-Propagacion:

- Cuando se lanza una excepcion con throw, JS busca el try-catch mas cercano en la pila
- Si no hay un catch en el ambito actual, la excepcion se propaga al metodo/funcion llamador y asi sucesivamente
- Si no se captura en ningun nivel el programa termina con un error, y en navegadores se muestra en consola
- En contextos asincronos, las excepciones no se propagan a menos que se usen dentro de una funcion async

Ejercicio 12: Sea el siguiente programa escrito en PYTHON:: Indique el camino de ejecución.

```
#!/usr/bin/env python
#calc.py
def dividir():
    x = a / b
    print (("Resultado"), (x))

while True:
    try:
        a = int(input("Ingresa el primer numero: \n"))
        b = int(input("Ingresa el segundo numero: \n"))
        dividir()
        break
```

```

except ZeroDivisionError:
    print ("No se permite dividir por cero")
finally:
    print ("Vuelve a probar")

```

a) Describa qué caminos ejecuta para diferentes valores de ingreso

1. Con cualquier valor numerico de a y un valor de b que no es 0:
 - No hay error de conversion ni division por 0
 - Se ejecuta dividir y muestra el resultado
 - Se ejecuta el finally "Vuelve a probar"
 - Finaliza la ejecucion del programa con el break
2. Con cualquier valor numerico de a y un valor de b que es 0:
 - Se ejecuta dividir
 - Ocurre ZeroDivisionError
 - Se ejecuta el bloque except ZeroDivisionError y muestra "No se puede dividir por cero"
 - Se ejecuta el bloque finally "Vuelve a probar"
 - Se reinicia el bucle
3. Con cualquier valor no numerico/caracter en ambas o una variable:
 - Ocurre ValueError
 - Se busca un except que trate esto y no se encuentra
 - Finaliza el programa con un error

b) Agregar el uso de una excepción anónima

```

#!/usr/bin/env python
#calc.py
def dividir():
    x = a / b
    print (("Resultado"), (x))

while True:

```

```
try:
    a = int(input("Ingresa el primer numero: \n"))
    b = int(input("Ingresa el segundo numero: \n"))
    dividir()
    break
except ZeroDivisionError:
    print ("No se permite dividir por cero")
except:
    print ("Error inesperado") # Aca se atrapa cualquier excepcion
finally:
    print ("Vuelve a probar")
```

Ejercicio 13: Sea el siguiente código escrito en JAVA

```
public class ExcepcionUno extends Exception {  
    public ExcepcionUno(){  
        super(); // constructor por defecto de Exception  
    }  
}
```

Hoja 3 de 7

Conceptos y Paradigmas de lenguajes de Programación 2026

```
    public ExcepcionUno( String cadena ){  
        super( cadena ); // constructor param. de Exception  
    }  
}  
  
public class ExcepcionDos extends Exception {  
    public ExcepcionDos(){  
        super(); // constructor por defecto de Exception  
    }  
  
    public ExcepcionDos( String cadena ){  
        super( cadena ); // constructor param. de Exception  
    }  
}  
  
public class ExcepcionTres extends Exception {  
    public ExcepcionTres(){  
        super(); // constructor por defecto de Exception  
    }  
  
    public ExcepcionTres( String cadena ){  
        super( cadena ); // constructor param. de Exception  
    }  
}
```

```

public class Lanzadora {

    public void lanzaSiNegativo(int param) throws ExcepcionUno {
        if (param < 0)
            throw new ExcepcionUno("Numero negativo");
    }

    public void lanzaSiMayor100(int param) throws ExcepcionDos {
        if (param >100 and param<125)
            throw new ExcepcionDos("Numero mayor100");
    }

    public void lanzaSiMayor125(int param) throws ExcepcionTres {
        if (param >= 125)
            throw new ExcepcionTres("Numero mayor125");
    }

}

```

```

import java.io.FileInputStream;
import java.io.IOException;

public class Excepciones {

    public static void main(String[] args) {
        // Para leer un fichero
    }
}

```

```

Lanzadora lanza = new Lanzadora();
FileInputStream entrada = null;
int leo;
try {
    entrada = new FileInputStream("fich.txt");
    while ((leo = entrada.read()) != -1){
        if (leo < 0)
            lanza.lanzaSiNegativo(leo);
        else if (leo > 100)
            lanza.lanzaSiMayor100(leo);
    }

    entrada.close();
    System.out.println("Todo fue bien");
}
catch (ExcepcionUno e) { // Personalizada
    System.out.println("Excepcion: " + e.getMessage());
}
catch (ExcepcionDos e) { // Personalizada
    System.out.println("Excepcion: " + e.getMessage());
}
catch (IOException e) { // Estándar
    System.out.println("Excepcion: " + e.getMessage());
}
finally {
    if (entrada != null)
        try {
            entrada.close(); // Siempre queda cerrado
        }
        catch (Exception e) {
            System.out.println("Excepcion: " + e.getMessage());
        }
    System.out.println("Fichero cerrado.");
}
}
}

```

a) Indique cómo se ejecuta el código. Debe quedar en claro los caminos posibles de ejecución, cuales son los manejadores que se ejecutan y cómo se buscan los mismos y si en algún caso se produce algún error.

1. El archivo fich existe, el primer valor leído es 20 y el segundo es -1:
 - a. No ocurren excepciones
 - b. Se cierra el archivo
 - c. Imprime "Todo fue bien"
 - d. Se ejecuta bloque finally
 - e. Imprime "Fichero cerrado"
 - f. Fin de la ejecución
2. El archivo fich existe, y el primer valor leído es -3:
 - a. Se llama a lanza.lanzaSiNegativo(leo)
 - b. Se lanza ExcepcionUno("Numero negativo")

- c. Se busca un manejador dinamicamente y se encuentra "catch(ExcepcionUno e)"
 - d. Imprime "Excepcion: Numero negativo"
 - e. Se ejecuta el bloque finally
 - f. Imprime "Fichero cerrado"
 - g. Fin de la ejecucion
3. El archivo fich existe y el primer valor leido es 101:
- a. Se llama a lanza.lanzaSiMayor100*(leo)
 - b. Se llama a ExcepcionDos("Numero mayor100")
 - c. Se busca un manejador dinamicamente y se encuentra "catch(ExcepcionDos e)"
 - d. Imprime "Excepcion: Numero mayor100"
 - e. Se ejecuta el bloque finally
 - f. Imprime "Fichero cerrado"
 - g. Fin de la ejecucion
4. El archivo fich existe y el primer valor leido es 200: (No estoy seguro de esta parte porque no aparece el manejo pero deberia ser asi)
- a. Se llama a lanza.lanzaSiMayor125(leo)
 - b. Se llama a ExcepcionTres("Numero mayor125")
 - c. Se busca un manejador dinamicamente y se encuentra "catch(ExceptionTres e)"
 - d. Se imprime "Excepcion: Numero mayor125"
 - e. Se ejecuta el bloque finally
 - f. Imprime "Fichero cerrado"
 - g. Fin de la ejecucion
5. El archivo fich no existe:
- a. Se lanza excepcion IOException
 - b. Se busca un manejador dinamicamente y se encuentra "catch(IOException e)"

- c. Se imprime "Excepcion ..."
 - d. Se ejecuta el bloque finally
 - e. Imprime "Fichero cerrado"
 - f. Fin de la ejecucion
6. El cierre del archivo falla:
- a. Se lanza una Excepcion
 - b. Se busca un manejador dinamicanete y se encuentra "catch(Exception e)"
 - c. Se imprime "Excepcion:"
 - d. Imprime "Fichero cerrado"
 - e. Fin de la ejecucion

Ejercicio 14: Dado el siguiente código en Java. Indique todos los posibles caminos de resolución, de acuerdo a los números que vaya leyendo del archivo.

```
class ExcepcionE1 extends Exception {  
    public ExcepcionE1(){  
        super(); // constructor por defecto de Exception  
    }  
    public ExcepcionE1( String cadena ){  
        super( cadena ); // constructor param. de Exception  
    }  
}
```



```

class ExcepcionE2 extends Exception {

    Public ExcepcionE2(){
        super(); // constructor por defecto de Exception
    }

    Public ExcepcionE2( String cadena ){
        super( cadena ); // constructor param. de Exception
    }

}

// Esta clase lanzará la excepción
public class Evaluacion {
    void Evalua( int edad ) throws ExcepcionE1, ExcepcionE2 {
        if ( edad < 18 )
            throw new ExcepcionE1( "Es una persona menor de edad" );
        else if ( edad > 70 )
            throw new ExcepcionE2( "Es persona mayor de edad" );

    }

    void Segmenta( int edad ) throws ExcepcionE1, ExcepcionE2 {
        if ( edad < 35 )
            throw new ExcepcionE1( "Es una persona joven" );

    }

}

```

```

class AnalisisEdadPoblacion{
    public static void main( String[] args ){
        // Para leer un fichero
        Evaluacion Invoca = new Evaluacion();
        FileInputStream entrada = null;
        int leo;
        try{
            entrada = new FileInputStream( "fich.txt" );
            while ( ( leo = entrada.read() ) != -1 ) {
                try {
                    if (leo<0) {
                        throw new ExcepcionE1( "Edad inválida" );
                    }
                    else{
                        if (leo>120){
                            throw new ExcepcionE1( "Edad inválida" );
                        }
                    }
                    invoca.evalua (leo);
                    invoca.segmenta( leo );
                    System.out.println( " ( "Es persona adulta, Todo fue bien" );
                }
                catch ( ExcepcionE2 e ){
                    System.out.println( "Excepcion: " + e.getMessage() );
                }
            }
        }
    }
}

```

Hc

Conceptos y Paradigmas de lenguajes de Programación 2026

```

    }
    catch ( ExcepcionE1 e ){
        System.out.println( "Excepcion: " + e.getMessage() );
    }
} catch (FileNotFoundException e1) {
    System.out.println("No se encontró el archivo");
} catch (IOException e) {
    System.out.println("Problema para leer los datos");
}finally {
    if (entrada != null)
        try {
            entrada.close();
        } catch (Exception e) {
            System.out.println("Excepcion: " + e.getMessage());
        }
    System.out.println("Fichero cerrado.");
}
}
}
}

```

Caminos:

1. Si se lee -1:

- a. Sale del while
 - b. Ejecute bloque finally
 - c. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos
2. Si se lee cualquier negativo (menos -1):
- a. Se lanza ExcepcionE1("Edad invalida")
 - b. Se busca estaticamente un manejador y se encuentra
 - c. Imprime el mensaje de la excepcion
 - d. Se ejecuta el finally
 - e. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos
3. Si se lee 121 o mas:
- a. Se lanza ExcepcionE1("Edad invalida")
 - b. Se busca estaticamente un manejador y se encuentra
 - c. Imprime el mensaje de la exepcion
 - d. Se ejecuta el finally
 - e. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos
4. Si se lee un valor entre 0 y 17:
- a. Se llama a invoca.evalua(leo)
 - b. Se llanza ExcepcionE1("Es una persona mayor de edad")
 - c. Se busca estaticamente un manejador y se encuentra
 - d. Imprime el mensaje de la excepcion
 - e. Se ejecuta el finally
 - f. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos
5. Si se lee un valor entre 18 y 34:
- a. Se llama a invoca.evalua(leo)
 - b. Se llama a invoca.segmenta(leo)

- c. Se lanza ExcepcionE1("Es una persona joven")
 - d. Se busca estaticametne un manejador y se encuentra
 - e. Imprime el mensaje de la excepcion
 - f. Se ejecuta el finally
 - g. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos
6. Si se lee un valor entre 35 y 70:
- a. Se llama a invoca.evalua(leo)
 - b. Se llama a inovca.segmenta(leo)
 - c. Se imprime "Es una persona adulta, Todo fue bien"
 - d. Se ejecuta el finally
 - e. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos
7. Si se lee un valor entre 71 y 121:
- a. Se llama a invoca.evalua(leo)
 - b. Se lanza ExcepcionE2("Es una persona mayor de edad")
 - c. Se busca un manejador estaticamente y se encuentra
 - d. Imprime el mensaje de la excepcion
 - e. Se ejecuta el finally
 - f. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos

Otros caminos si no pasan los anteriores:

8. El archivo no existe:
- a. Se lanza FileNotFoundException
 - b. Se busca un manejador y no se encuentra
 - c. Imprime "No se encontro el archivo"
 - d. Se ejecuta bloque finally
 - e. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos

9. Problema para leer los datos:

- a. Se lanza IOException
- b. Se busca un manejador estaticamente y se encuentra
- c. Imprime "Problema para leer los datos"
- d. Se ejecuta el bloque finally
- e. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos

10. Problemas para cerrar el archivo en el finally:

- a. Se lanza Exception
- b. Se busca un manejador estaticamente y se encuentra
- c. Imprime el mensaje de la excepcion
- d. Sigue leyendo hasta el siguiente valor del archivo y finaliza si no hay mas datos